

Архитектурни шаблони

доц. д-р Емил Дойчев

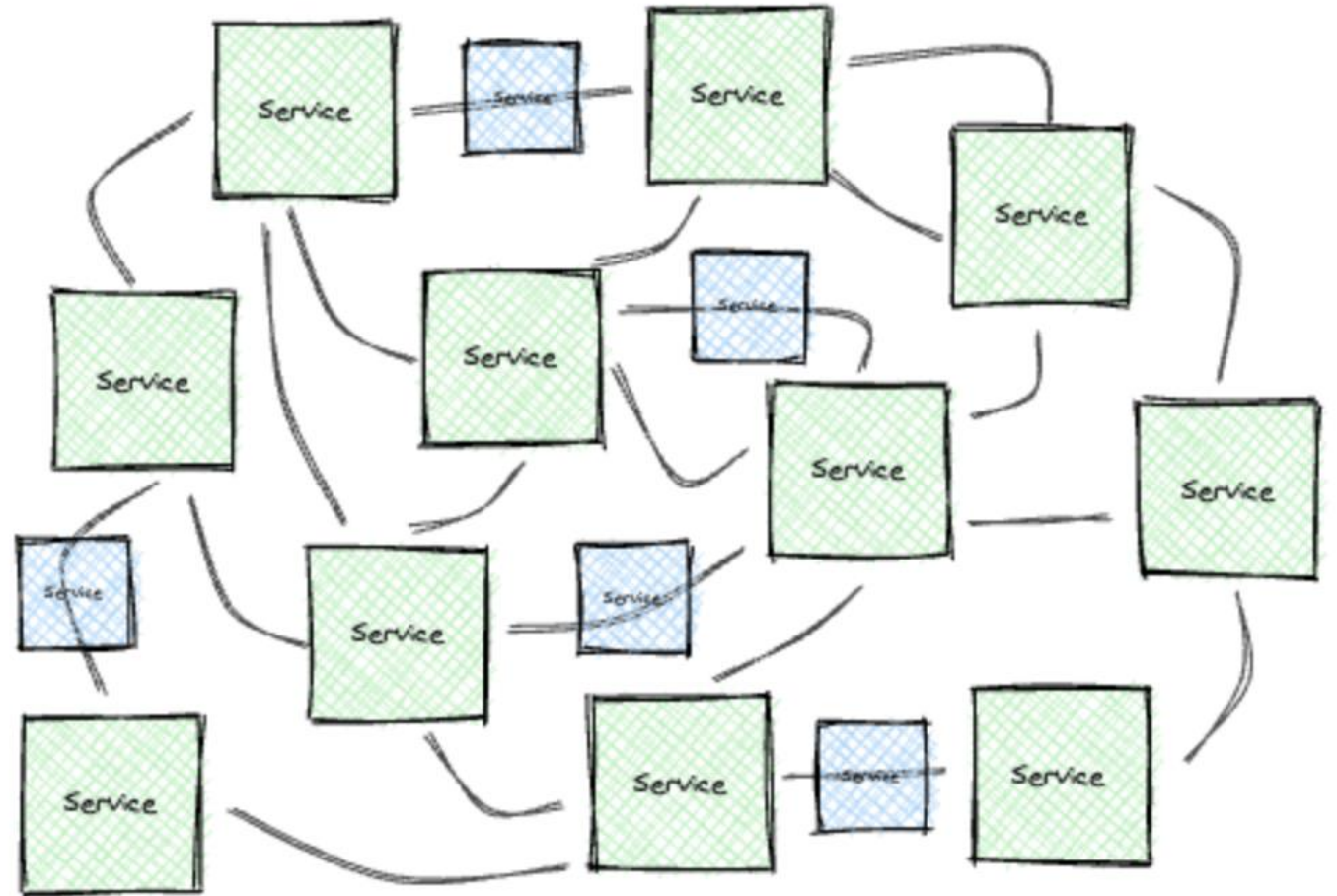
Магистърска програма „Софтуерно инженерство и технологии“

Мотивация

- Често се случва разработчиците да започнат работа по приложение без да имат ясна архитектура.
- Без ясна и добре дефинирана архитектура повечето разработчици и архитекти прибягват до де факто стандартния традиционен модел на многослойна архитектура (n-tier архитектура), като създават неявни слоеве чрез разделяне на модулите на изходния код в пакети.
- За съжаление тази практика често води до набор от неорганизирани модули от сорс код, които нямат ясни роли, отговорности и взаимоотношения помежду си.
- Това обикновено се нарича антимоделът на архитектурата "голяма топка кал" (big ball of mud architecture anti-pattern).



Big Ball of Mud





Мотивация

- Приложенията, които нямат формална архитектура, обикновено са изградени от силно свързани модули, крехки са, трудни са за промяна и са без ясна визия или посока.
 - В резултат на това е много трудно да се определят архитектурните характеристики на приложението, без да се разбере напълно вътрешната работа на всеки компонент и модул в системата:
 - Мащабира ли се архитектурата?
 - Каква е производителността на приложението?
 - Колко лесно приложението реагира на промени?
 - Какви са възможностите за инсталиране на приложението?
 - Доколко архитектурата е адаптивна?
-



Мотивация

- Архитектурните шаблони помагат да се определят основните характеристики и поведение на дадено приложение.
 - Например някои архитектурни модели са предназначени за силно мащабируеми приложения, докато други архитектурни модели са предназначени за приложения, които е необходимо да са много гъвкави.
 - Познаването на характеристиките, силните и слабите страни на всеки архитектурен модел е необходимо, за да се избере този, който отговаря на специфичните бизнес нужди и цели.
-



Многослойна архитектура

Layered Architecture

Това е най-разпространеният архитектурен модел. Той е де факто стандарт за повечето Enterprise Java приложения, поради което е широко известен на повечето архитекти, дизайнери и разработчици.

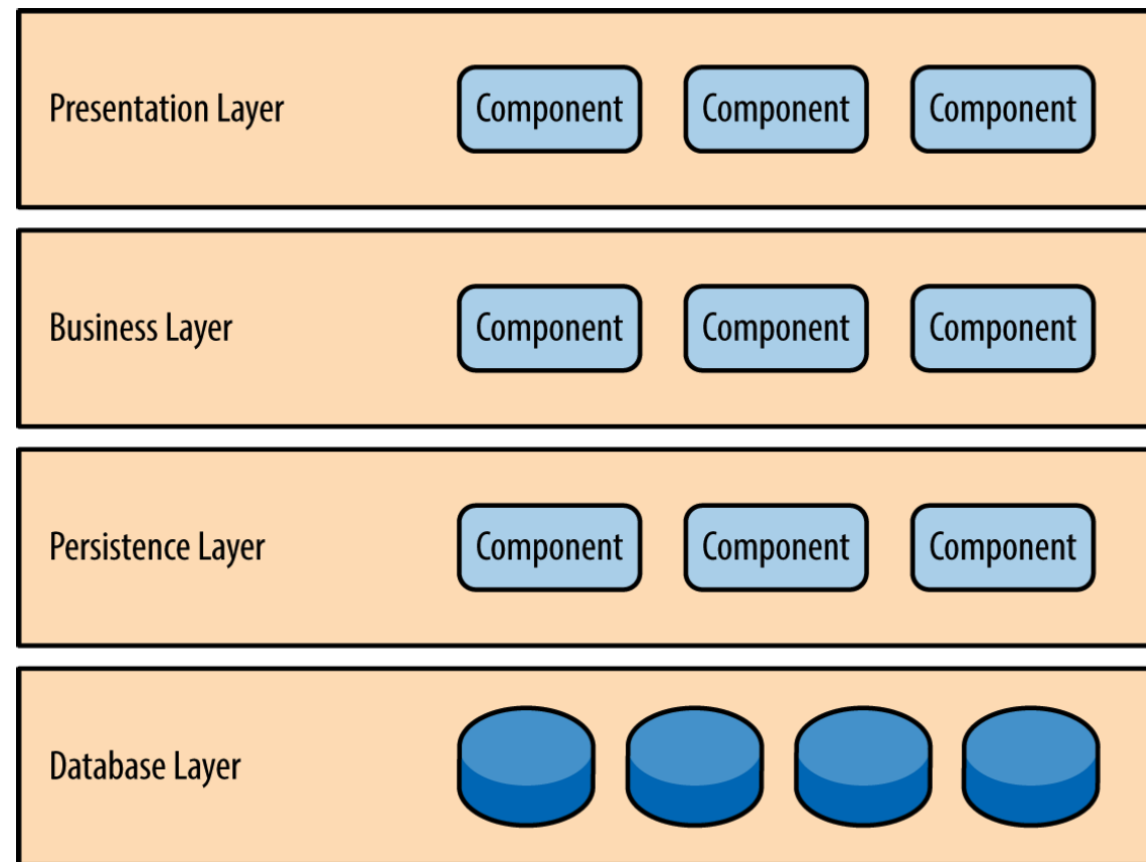
Моделът е близък до традиционните структури на ИТ комуникация и организация, които се срещат в повечето компании, което го прави естествен избор при разработването на повечето бизнес приложения.



Многослойна архитектура

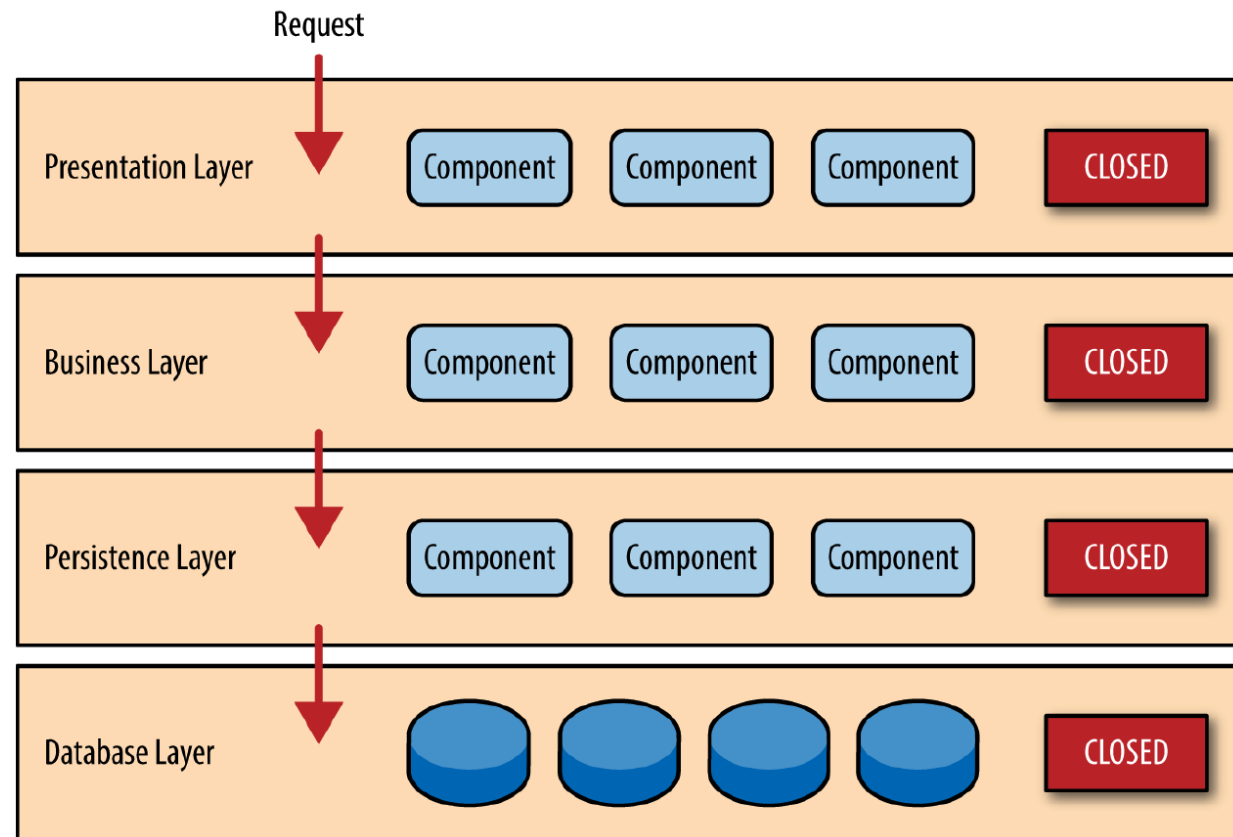
- Компонентите в модела на многослойната архитектура са организирани в хоризонтални слоеве, като всеки слой изпълнява специфична роля в приложението (например потребителски интерфейс или бизнес логика).
- Въпреки че моделът за многослойна архитектура не определя броя и видовете слоеве, които трябва да съществуват в модела, повечето многослойни архитектури се състоят от четири стандартни слоя:
 - Presentation
 - Business
 - Persistence
 - Database

Многослойная архитектура. Роли.



Многослойна архитектура

- Всеки от слоевете в архитектурата е отбелязан като затворен. Това е много важна концепция в модела на многослойната архитектура.
- Затвореният слой означава, че когато една заявка се придвижва от слой към слой, тя трябва да премине през слоя точно под него, за да стигне до следващия слой под него.





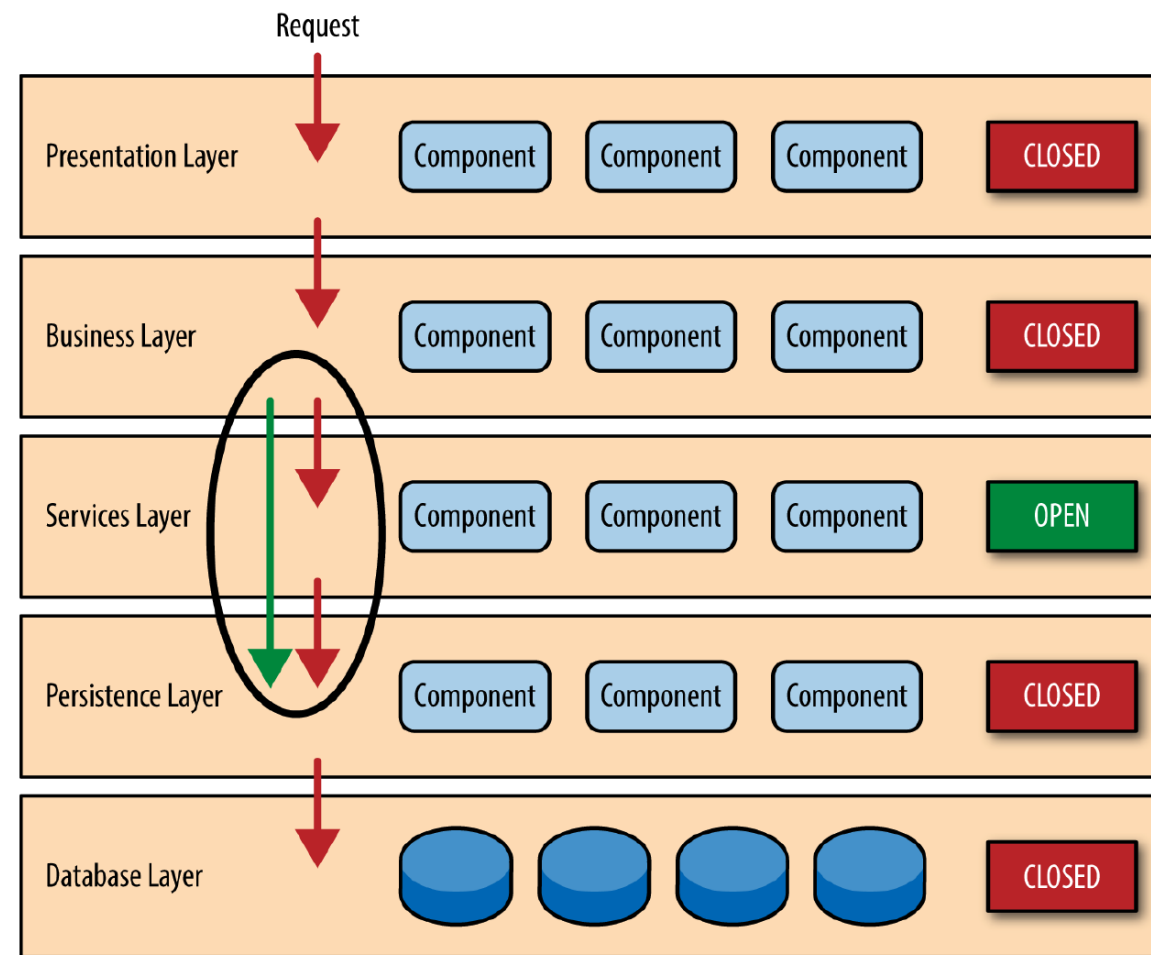
Изолиране на слоевете

- Концепцията за изолация на слоевете означава, че промените, направени в един от слоевете на архитектурата, обикновено не засягат компонентите в други слоеве - промяната е изолирана за компонентите в този слой и евентуално за друг свързан слой.
 - В противен случай се получават твърде много връзки между слоеве, които не са директни „съседни“, което води до силна свързаност с много зависимости между отделните компоненти.
-

Отворени слоеве

- Има случаи, в които е смислено да има „отворени“ слоеве, които нарушават принципа на изолацията.
- Например, ако трябва да се добави слой за споделени услуги към архитектурата, който съдържа общи компоненти, до които имат достъп компонентите в бизнес слоя. Създаването на слой за услуги обикновено е добра идея в този случай, тъй като от архитектурна гледна точка то ограничава достъпа до споделените услуги само от бизнес слоя (а не от presentation слоя). Без отделен слой няма нищо, което архитектурно да ограничава достъпа на презентационния слой до тези общи услуги.
- Това обаче е проблем, тъй като бизнес слойът трябва да премине през слоя на споделените услуги, за да стигне до persistence слоя, което е безсмислено.
- Това е стар проблем с многослойната архитектура, който се решава чрез създаване на отворени слоеве в архитектурата.

Многослойна архитектура. Отворени слоеве.



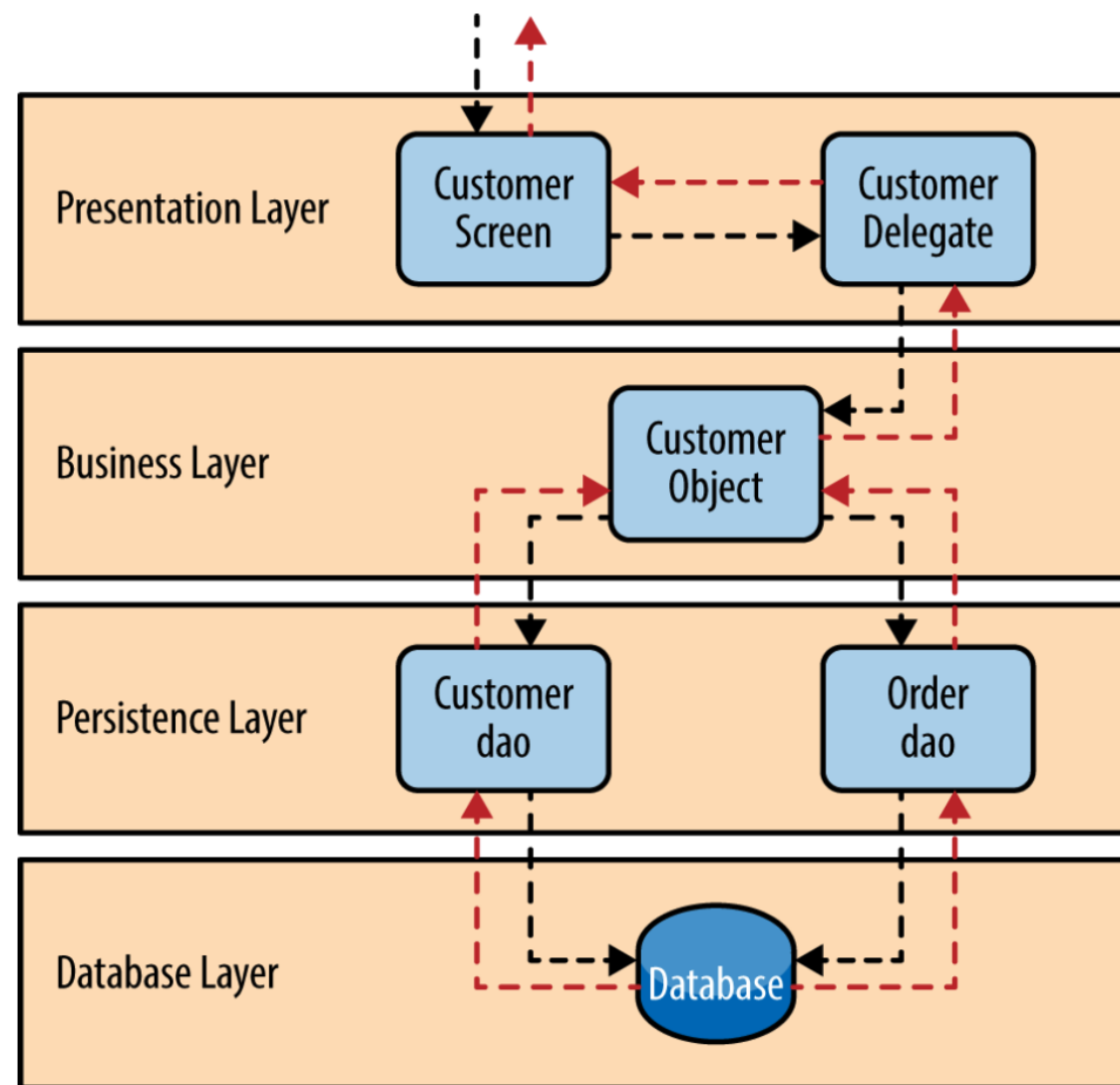


Отворени слоеве

- Използването на концепцията за отворени и затворени слоеве помага да се определи връзката между слоевете на архитектурата и потоците от заявки, а също така предоставя на проектантите и разработчиците необходимата информация, за да разберат различните ограничения за достъп до слоевете в рамките на архитектурата.
 - Ако не се документира правилно кои слоеве в архитектурата са отворени и затворени (и защо), това ще доведе до тясно свързани и крехки архитектури, които са много трудни за тестване, поддръжка и внедряване.
-

Многослойна архитектура. Пример.

- Примерът разглежда заявка от потребител за извличане на клиентската информация за определено лице. Черните стрелки показват, че заявката се насочва към базата данни, за да се извлекат данните за клиента, а червените стрелки показват, че отговорът се насочва обратно към екрана, за да се покажат данните.
- В този пример информацията за клиента се състои както от данни за клиента, така и от данни за поръчките (поръчки, направени от клиента).



Съображения

- Многослойната архитектура е солиден модел с общо предназначение, което го прави добра отправна точка за повечето приложения, особено когато няма яснота кой архитектурен модел е най-подходящ за приложението.
- Едно от нещата, за които трябва да се внимава, е т.нар. антимодел "архитектурна дупка" (*architecture sinkhole anti-pattern*).
 - Този антимодел описва ситуацията, в която заявките преминават през няколко слоя на архитектурата с малко или никаква логика, извършвана във всеки слой.
 - Типично е около 20% от заявките да преминават без обработка през слоевете, а 80% от заявките да имат някаква бизнес логика, свързана със заявката.
- Друго съображение при модела на многослойната архитектура е, че той е склонен да води към създаване на монолитни приложения, дори ако се раздели слоя за представяне и бизнес слоя на модули, които се инсталират по отделно.
 - Въпреки че това може да не е проблем за някои приложения, то създава някои потенциални проблеми по отношение на внедряването, общата устойчивост и надеждност, производителността и мащабируемостта.



Събитийно-ориентирана архитектура

Event-Driven Architecture

Архитектурният модел, управляван от събития, е популярен разпределен асинхронен архитектурен модел, използван за създаване на силно мащабируеми приложения. Той също така е с висока степен на адаптивност и може да се използва както за малки, така и за големи и сложни приложения. Архитектурата, управлявана от събития, е съставена от изолирани, едноцелеви компоненти за обработка на събития, които работят асинхронно.

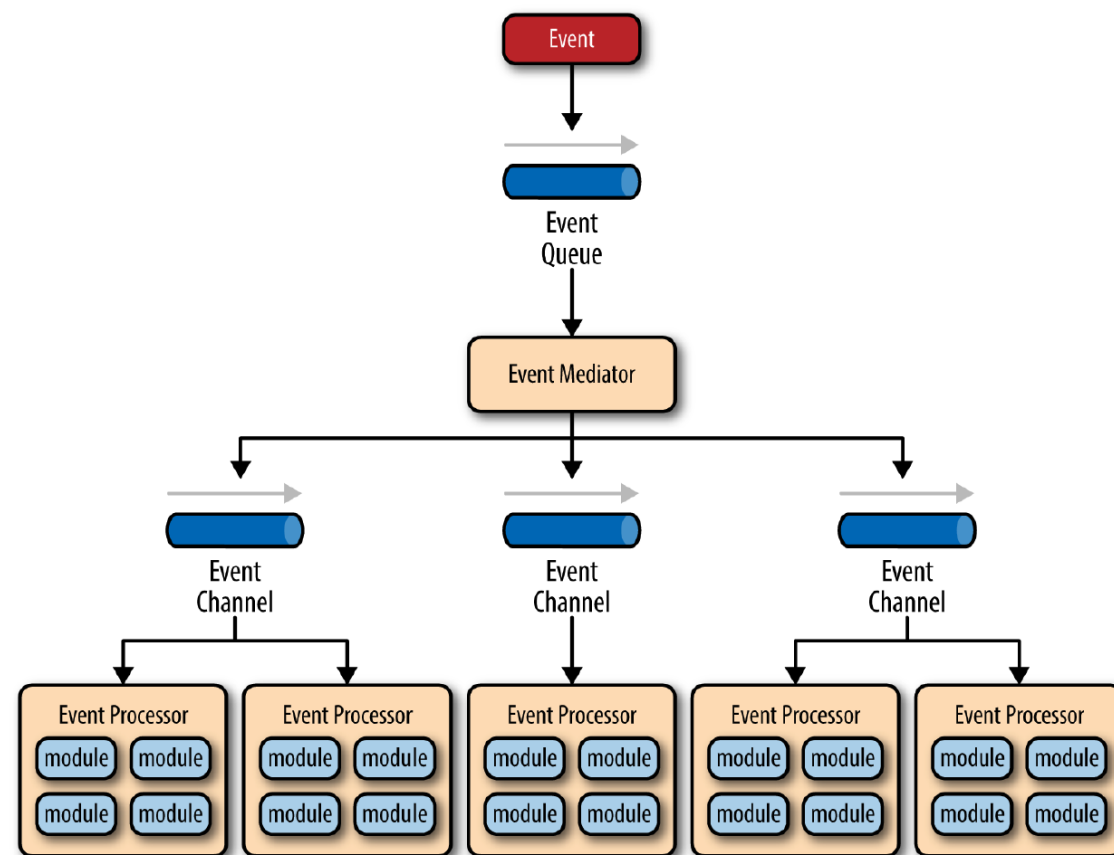
Моделът се състои от две основни топологии: Посредник и Брокер.



Събитийно-ориентирана архитектура. Топология „Посредник“

Mediator Topology

Топологията “Посредник” е подходяща за събития, които имат няколко стъпки и изискват определено ниво на оркестрация за обработка на събитието. Например, единично събитие за сключване на борсова сделка може да изисква първо да се валидира сделката, след това да се провери съответствието на тази борсова сделка с различни ограничения, да се възложи сделката на брокер, да се изчисли комисионната и накрая да се сключи сделката при този брокер. Всички тези стъпки ще изискват определено ниво на оркестрация, за да се определи редът на стъпките и кои от тях могат да се извършват последователно, и кои паралелно.



Топология „Посредник“

- В топологията има четири основни типа архитектурни компоненти:
 - опашки за събития (event queues)
 - посредник на събития (event mediator)
 - канали за събития (event channels)
 - процесори за събития (event processors)
 - Потокът от събития започва с изпращане на събитие от клиент към *опашка за събития*, която се използва за пренасяне на събитието към *посредника на събития*.
 - *Посредникът* получава първоначалното събитие и го организира (оркестрира), като изпраща допълнителни асинхронни събития към *каналите за събития*, за да изпълни всяка стъпка от процеса.
 - *Процесорите на събития*, които слушат каналите за събития, получават събитието от посредника и изпълняват специфична бизнес логика за обработка на събитието.
-

Топология „Посредник“

- Обикновено в архитектура, управлявана от събития, има от десетки до няколкостотин **опашки за събития (event queues)**. Шаблонът не уточнява реализацията на компонента на опашката за събития - той може да бъде:
 - Message Queue
 - Web Service Endpoint
 - или всякаква комбинация от тях.
 - В този модел има два вида събития:
 - начално събитие (initial event)
 - събитие за обработка (processing events).
 - Първоначалното събитие е оригиналното събитие, получено от посредника, докато събитията за обработка са тези, които се генерират от посредника и се получават от компонентите за обработка на събития.
-



Топология „Посредник“

- **Каналите за събития (event channels)** се използват от посредника за асинхронно предаване на специфични събития за обработка към процесорите за събития.
 - Каналите за събития могат да бъдат или *message queues*, или *message topics*.
 - Message Topics се използват най-широко с топологията “Посредник”, за да може събитията за обработка да се получават от множество процесори, всеки от които изпълнява различна функционалност.
-



Топология „Посредник“

- **Компонентите за обработка на събития (event processors)** съдържат бизнес-логиката на приложението.
 - Те са самостоятелни, независими, и изолирани архитектурни компоненти, които изпълняват конкретна задача в приложението или системата.
 - Въпреки че имплементацията на тези компоненти може да варира от твърде фина (например просто изчисляване на данък върху продажбите на поръчка) до твърде груба (например пълна обработка на застрахователна претенция), важно е да се има предвид, че като цяло всеки компонент на процесора на събития трябва да изпълнява една бизнес задача и да не разчита на други процесори на събития за изпълнение на конкретната си задача.
-

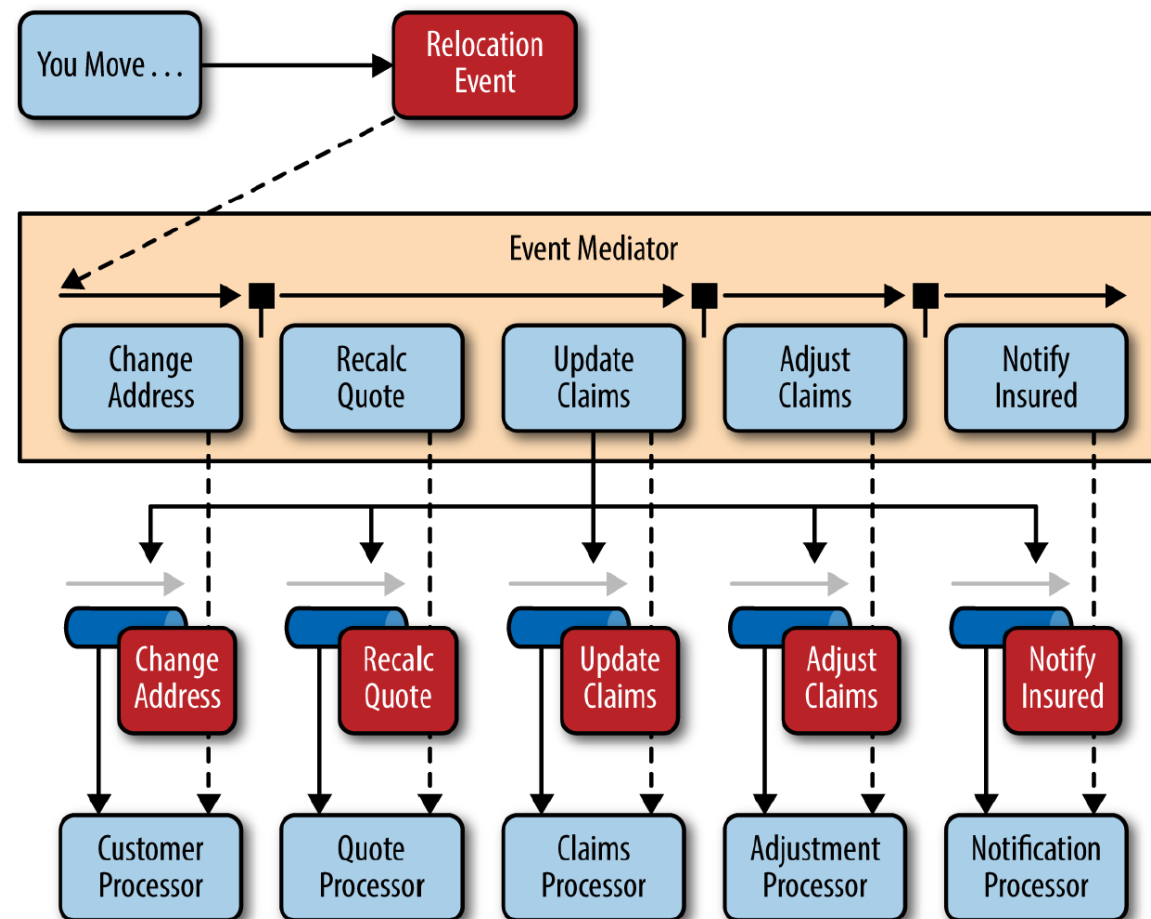
Топология „Посредник“

- **Посредникът на събития** може да бъде реализиран по различни начини:
 - Най-простата и най-често срещана имплементация е чрез интеграционни хъбове с отворен код, като Spring Integration, Apache Camel или Mule ESB. Потоците от събития в тези интеграционни хъбове обикновено се реализират чрез Java код или DSL (език, специфичен за домейна).
 - За по-сложна медиация и оркестрация може да се използва BPEL (business process execution language), съчетан с BPEL engine, като например Apache ODE (с отворен код). BPEL е стандартен XML-подобен език, който описва необходимите данни и стъпки за обработка на едно първоначално събитие.
 - За много големи приложения, изискващи много по-сложна оркестрация (включително стъпки, включващи намесата на човек), посредника на събития може да се реализира като се използва мениджър на бизнес процеси (BPM), например jBPM.

Топология „Посредник“.

Пример.

В примера се предполага, че клиент е застрахован в една застрахователна компания и иска да се премести в друга. В този случай първоначалното събитие може да се нарече „събитие за преместване“. Стъпките, свързани с обработката на това събитие, се съдържат в посредника. За всяка стъпка на първоначалното събитие посредникът създава събитие за обработка (например, промяна на адреса, преизчисляване на офертата и т.н.), изпраща това събитие за обработка към канала за събития и изчаква събитието за обработка от съответния процесор на събития (например, обработка на клиента, обработка на офертата и т.н.). Този процес продължава, докато всички стъпки в първоначалното събитие бъдат обработени. Общата стрелка над стъпките Recalc Quote и Update Claims в посредника показва, че тези стъпки могат да се изпълняват паралелно.



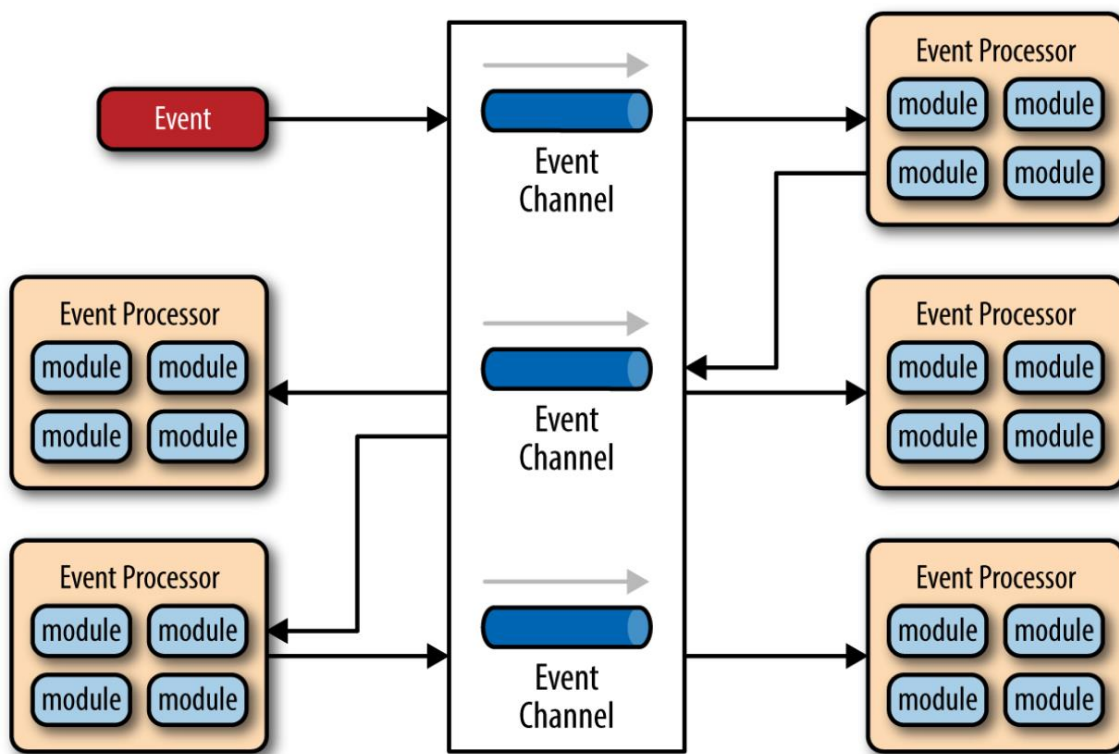
Събитийно-ориентирана архитектура.

Топология „Брокер“

Broker Topology

Топологията „Брокер“ се различава от топологията „Посредник“ по това, че няма централен медиатор на събития. Потокът от съобщения се разпределя между компонентите обработчици на събития по верижен начин чрез лек брокер на съобщения (напр. ActiveMQ, RabbitMQ, HornetQ и др.).

Тази топология е полезна, когато е на лице сравнително прост поток за обработка на събития и не е необходима централна оркестрация на събития.





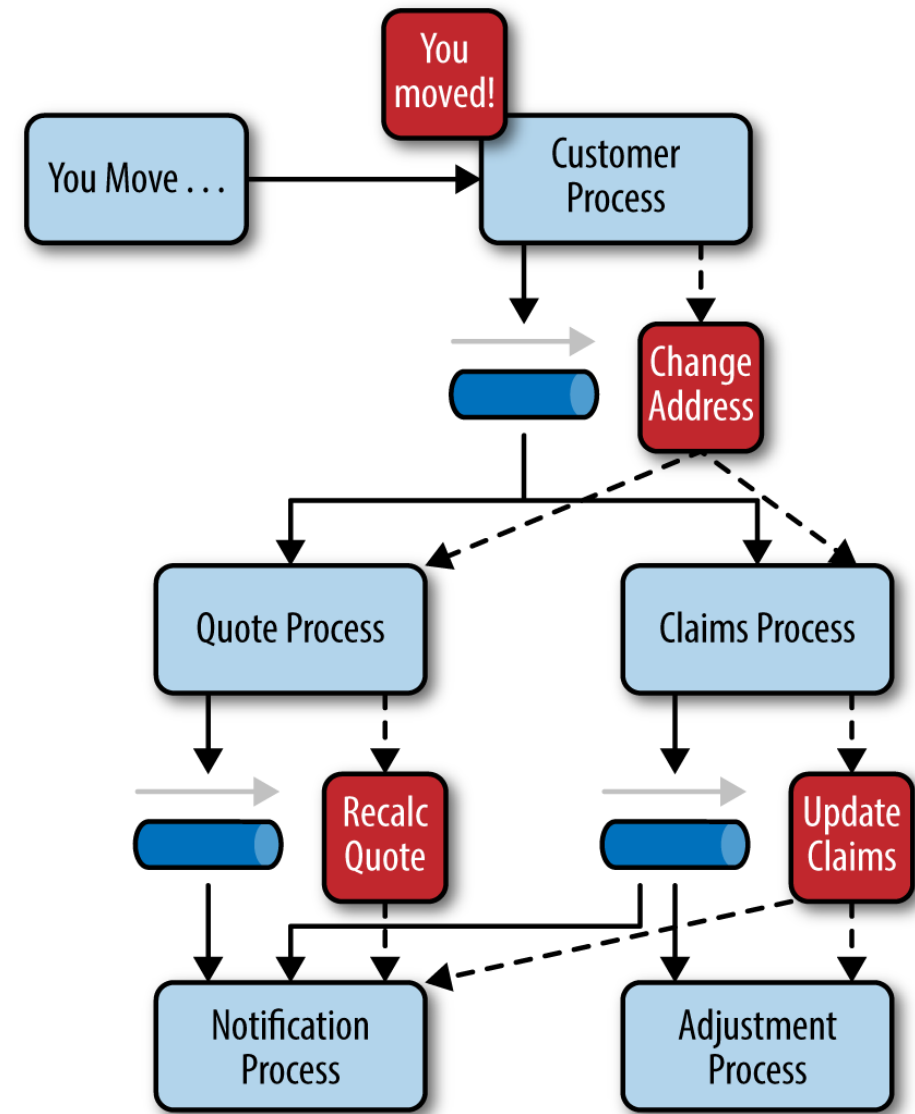
Топология „Брокер“

- Има два основни типа архитектурни компоненти в рамките на тази топология:
 - Компонент брокер
 - Компонент за обработка на събития.
 - Компонентът „Брокер“ може да бъде или централизиран или федерация от брокери, и съдържа всички канали за събития, които се използват в потока от събития.
 - Каналите за събития, съдържащи се в брокера, могат да бъдат:
 - Message Queues
 - Message Topics
 - или комбинация от двете
-

Топология „Брокер“.

Пример.

Тъй като в топологията „Брокер“ няма централен посредник на събития, който да получи първоначалното събитие, компонентът Customer Process получава събитието директно, променя адреса на клиента и изпраща съобщение, че е променил адреса на клиента (*change address event*). В този пример има два процесора на събития, които се интересуват от събитието за промяна на адреса: Quote Process и Claims Process. Първият компонент преизчислява новите застрахователни проценти въз основа на промяната на адреса и публикува събитие към останалата част от системата, посочващо какво е направил (*recalc quote event*). Компонентът за обработка на претенции, от друга страна, получава същото събитие за промяна на адреса, но в този случай той актуализира съществуваща застрахователна претенция и публикува събитие в системата като събитие за актуализация на претенция (*update claims event*). След това тези нови събития се приемат от други компоненти за обработка на събития и веригата от събития продължава в системата, докато не се публикуват повече събития за конкретното инициращо събитие.



Съображения

- Шаблонът за архитектура, управлявана от събития, е сравнително сложен за изпълнение, главно поради асинхронния си разпределен характер.
- При изпълнението на този шаблон трябва да се обърне внимание на различни проблеми на разпределената архитектура, като например:
 - наличност на отдалечени процеси
 - липса на отговор
 - функционалност за възстановяване на разпадната връзка.
- Едно от съображенията, които трябва да се вземат предвид при избора на този архитектурен модел, е липсата на атомични транзакции за единичен бизнес-процес.
 - Тъй като компонентите на процесора за събития са силно изолирани и разпределени, е много трудно да се поддържа обща транзакция. Ако се установи, че трябва да се раздели една единица работа между процесорите за събития, т.е. ако се използват отделни процесори за нещо, което би трябвало да бъде неделима транзакция, вероятно този модел не е подходящ за това приложение.
- Един от най-трудните аспекти на архитектурния модел, базиран на събития, е създаването, поддържането и управлението на договорите за компонентите на процесора на събития.
 - Всяко събитие обикновено има специфичен договор, свързан с него (напр. стойностите на данните и формата на данните, които се предават на процесора на събития).
 - Изключително важно е при използването на този модел да се избере стандартен формат на данните (например XML, JSON, Java Object и т.н.) и да се установи политика на управление на версиите на договорите от самото начало.



Архитектура от микроуслуги

Microservices Architecture

Архитектурният модел на микроуслугите бързо набира популярност в индустрията като адекватна алтернатива на монолитните приложения и архитектурите, ориентирани към услуги. Тъй като този архитектурен модел все още се развива, в индустрията има много неясноти относно това какво представлява този модел и как се прилага.





Архитектура с микросървиси

- Независимо от избраната топология или стил на реализация, има няколко общи основни концепции, които се отнасят до общия архитектурен модел.
 - Първата от тези концепции е понятието за *компоненти, които се внедряват отделно*. Това позволява по-лесно внедряване чрез ефективна и рационализирана линия за доставка (pipeline), повишена мащабируемост и висока степен на разделяне на приложенията и компонентите.
-

Архитектура с микросървиси

- Най-важната концепция, която трябва да се разбере при този модел, е понятието за **компонент на услугата**.
- Вместо да се мисли за услуги в архитектурата на микроуслугите, по-добре е да се мисли за *компоненти на услуги*, които могат да варират по степен на детайлност от един модул до голяма част от приложението.
- Компонентите на услуги съдържат един или повече модули (например Java класове), които представляват или функция с единично предназначение (например предоставяне на информация за времето за определен град или населено място), или независима част от голямо бизнес приложение (например, сключване на борсови сделки или сключване на автомобилни застраховки).
- Проектирането на правилното ниво на гранулярност на компонентите на услугите е едно от най-големите предизвикателства в архитектурата на микросървисите.

Архитектура с микросървиси

- Друга ключова концепция в архитектурния модел на микроуслугите е, че това е разпределена архитектура, което означава, че всички компоненти в архитектурата са напълно отделени един от друг и са достъпни чрез някакъв протокол за отдалечен достъп (например JMS, AMQP, REST, SOAP, RMI и др.).
- Разпределеният характер на този архитектурен модел е начинът, по който той постига някои от своите силни характеристики на мащабируемост и внедряване.
- Едно от най-интересните неща за архитектурата на микроуслугите е, че тя се е развила заради проблеми, свързани с други общи архитектурни модели, а не е създадена като решение, което чака да възникне проблем.
- Архитектурният стил на микросървисите се е развил естествено от два основни източника: монолитни приложения, разработени с помощта на модела на многослойната архитектура, и разпределени приложения, разработени с помощта на модела на архитектурата, ориентирана към услуги.

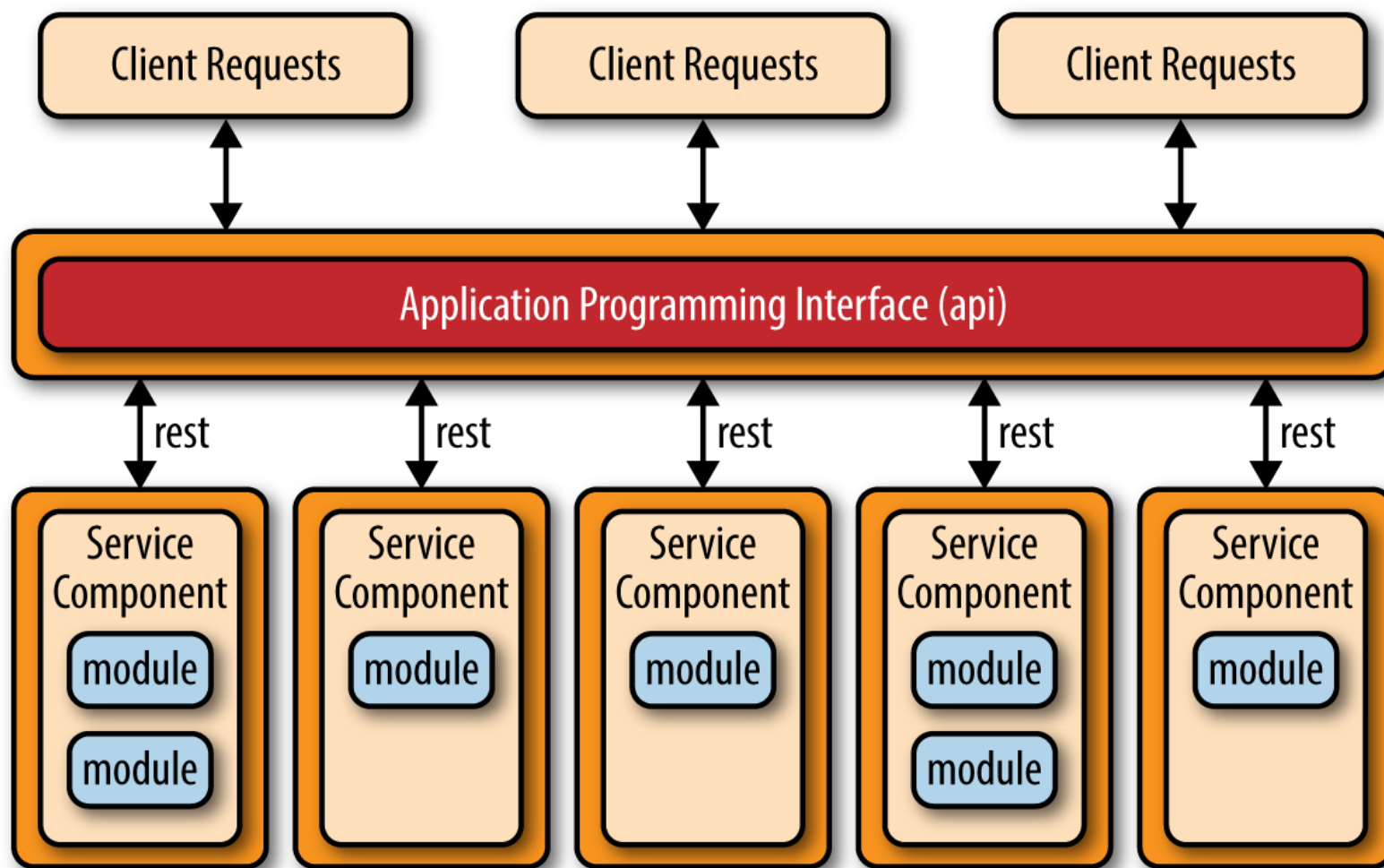
Архитектура с микросървиси

- Еволюционният път от монолитни приложения към стил на архитектура на микросървиси е продиктуван главно от развитието на непрекъснатата доставка (continuous delivery), понятието за непрекъснат процес на внедряване (continuous deployment) от разработката до производството, които улесняват внедряването на приложенията.
- Монолитните приложения обикновено се състоят от тясно свързани компоненти, които са част от единствена единица за внедряване, което прави промените, тестването и внедряването на приложението тромави и трудни (оттук и появата на обичайните цикли на "месечно внедряване", които се срещат в повечето големи ИТ компании).
- Тези фактори обикновено водят до крехки приложения, които се счупват всеки път, когато се внедрява нещо ново. Архитектурният модел на микроуслугите решава тези проблеми, като разделя приложението на множество отделни единици (компоненти на услуги), които могат да бъдат индивидуално разработени, тествани и внедрени независимо от други компоненти на услуги.



Архитектура с микросървиси

- Другият еволюционен път, който води до архитектурния модел на микроуслугите, е свързан с проблеми, открити при приложенията, прилагащи ориентираните към услуги модели (SOA).
 - Въпреки че моделът SOA е много мощен и предлага несравними нива на абстракция, хетерогенна свързаност, оркестрация на услуги и обещание за съгласуване на бизнес целите с възможностите на ИТ, той все пак е сложен, скъп, труден за разбиране и внедряване и обикновено е излишен за повечето приложения.
 - Стилът на архитектурата на микроуслугите се справя с тази сложност, като опростява смисъла на услугата, елиминира нуждите от оркестрация и опростява свързаността и достъпа до компонентите на услугата.
-



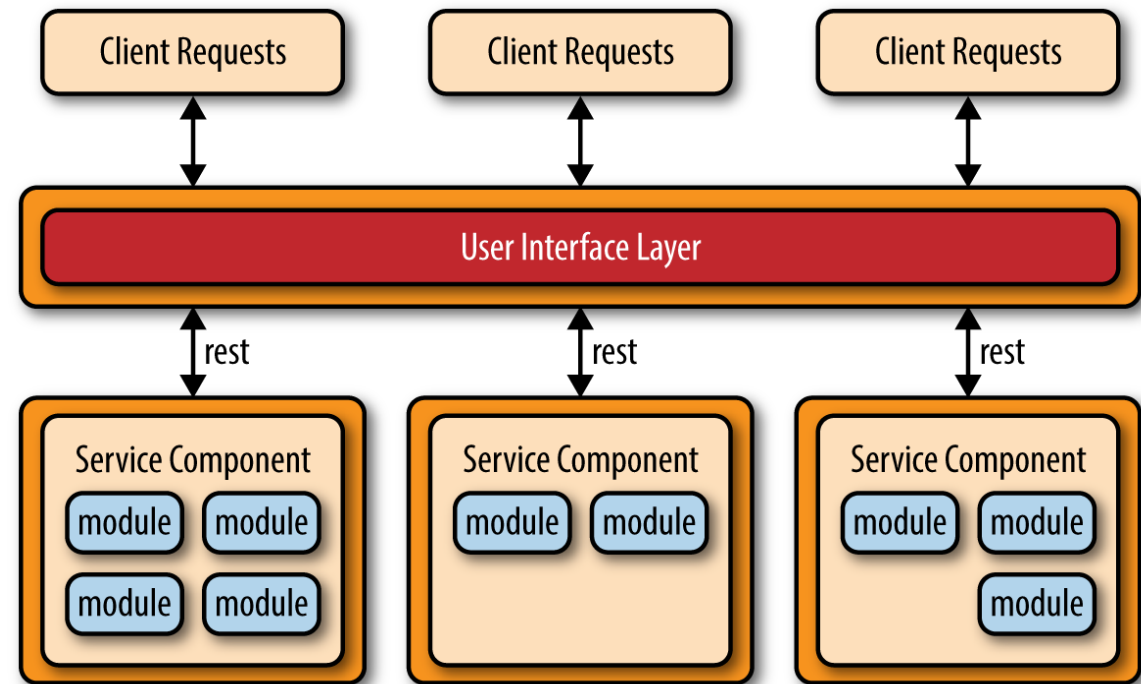
Микроуслуги.
Топология
„API REST“



Топология API-REST

- Топологията, базирана на API REST, е полезна за уебсайтове, които предлагат малки, самостоятелни индивидуални услуги чрез някакъв API.
 - Тази топология се състои от компоненти на услуги с много фина структура (откъдето идва и името микроуслуги), които съдържат един или два модула, изпълняващи специфични бизнес функции, независими от останалите услуги.
 - В тази топология тези компоненти на услугите обикновено се достъпват чрез REST-базиран интерфейс, реализиран чрез отделно внедрен уеб базиран API слой.
 - Примерите за тази топология включват някои от обичайните облачни REST-базирани уеб услуги, предлагани от Yahoo, Google и Amazon.
-

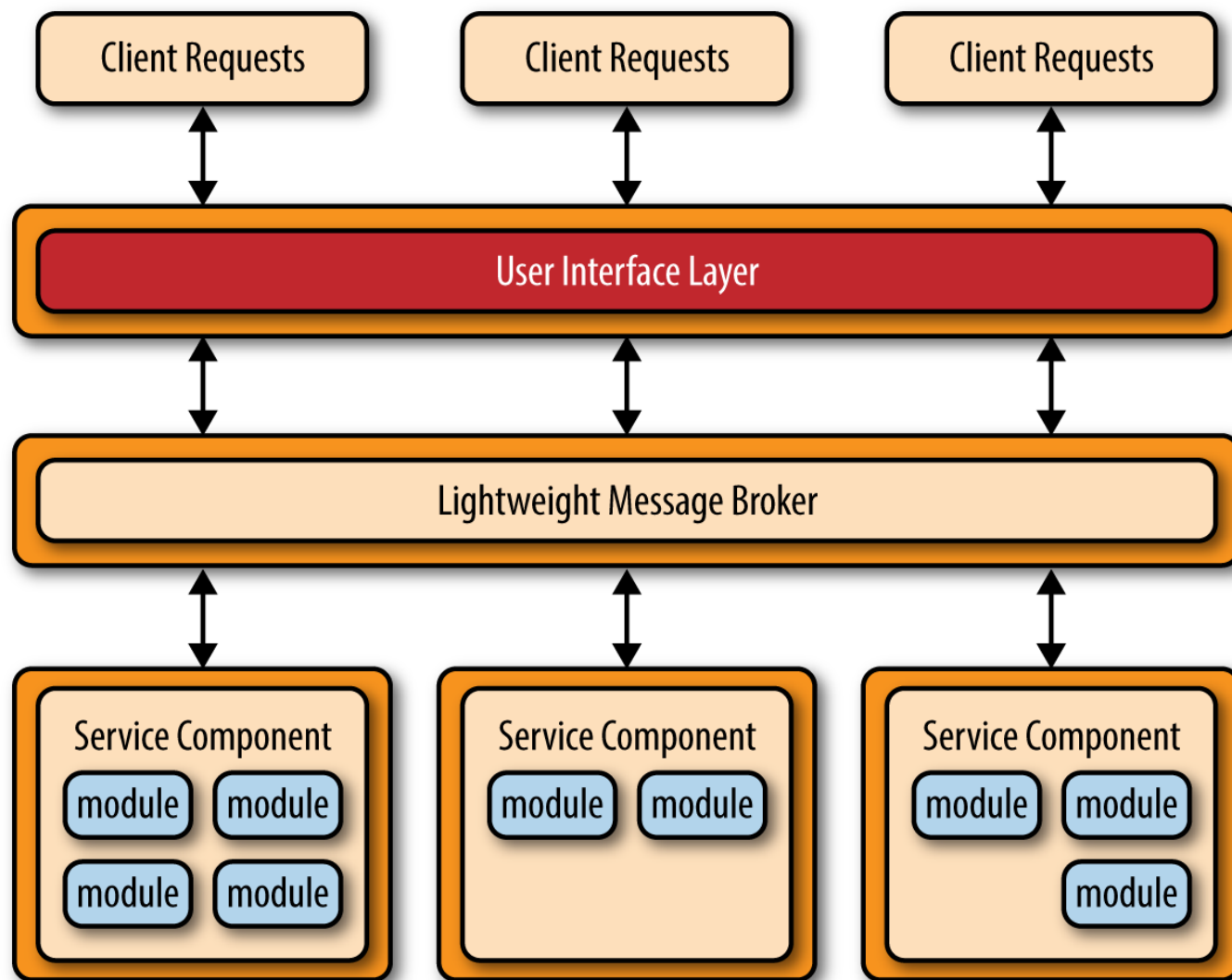
Микроуслуги. Топлогия „Application REST”





Топология Application REST

- Топологията, базирана на application REST, се различава от подхода, базиран на API REST, по това, че клиентските заявки се получават по-скоро чрез традиционните екрани на бизнес приложения, базирани на уеб или дебел клиент, отколкото чрез прост слой API.
 - Сложат на потребителският интерфейс на приложението е разположен като отделно уеб приложение, което осъществява отдалечен достъп до отделно разположени компоненти на услуги (бизнес функционалност) чрез прости REST-базирани интерфейси.
 - Компонентите на услуги в тази топология се различават от тези в API-REST-базираната топология по това, че тези компоненти на услуги са по-скоро по-големи, по-груби и представляват малка част от цялостното бизнес приложение, отколкото фино-размерни, единични услуги.
 - Тази топология е обичайна за малки и средни бизнес приложения, които имат сравнително ниска степен на сложност.
-



Микроуслуги.
Топология
„Centralized
messaging”

Топлогия „Centralized messaging”

- Друг често срещан подход в рамките на архитектурата на микроуслугите е топологията на централизираните съобщения.
- Тази е подобна на предишната REST-базирана топология на приложенията, с изключение на това, че вместо да използва REST за отдалечен достъп, тази топология използва лек централизиран брокер на съобщения (например ActiveMQ, HornetQ и др.).
- Изключително важно е, когато се разглежда тази топология, да не се бърка с модела на архитектура, ориентирана към услуги, или да се счита за "SOA-Lite".
- Лекият брокер на съобщения, намиращ се в тази топология, не извършва никаква оркестрация, трансформация или сложна маршрутизация. Той е просто лек транспорт за достъп до отдалечени компоненти на услуги.

Топлогия „Centralized messaging”

- Топологията на централизираните съобщения обикновено се среща в по-големи бизнес приложения или приложения, изискващи по-сложен контрол върху транспортния слой между потребителския интерфейс и компонентите на услугата.
- Предимствата на тази топология в сравнение с обикновената REST-базирана топология, са усъвършенствани механизми за поддръждане на опашки, асинхронни съобщения, мониторинг, обработка на грешки и по-добро цялостно балансиране на натоварването и мащабируемост.
- Въпросите, свързани с единичната точка на повреда (single point of failure) и архитектурното тясно място, които обикновено се свързват с централизирания брокер, се решават чрез клъстериране на брокери и федерация на брокери (разделяне на една инстанция на няколко инстанции, за да се разпредели натоварването на пропускателната способност на съобщенията въз основа на функционалните области на системата).



Избягване на зависимости и оркестрация

- Едно от основните предизвикателства на архитектурния модел на микроуслугите е определянето на правилното ниво на детайлност за компонентите на услугите.
 - Ако компонентите на услугите са твърде груби, може да не могат да се реализират ползите, които идват с този архитектурен модел (разгръщане, мащабируемост, тестване и свободно свързване).
 - Ако обаче компонентите на услугата са твърде дребнозърнести, това ще доведе до изисквания за оркестрация на услугите, което бързо ще превърне стройната архитектура на микроуслугите в тежка архитектура, ориентирана към услугите, пълна с цялата сложност, объркване, разходи и излишъци, които обикновено се срещат при приложенията, базирани на SOA.
-



Избягване на зависимости и оркестрация

- Ако се установи, че трябва да се организират компонентите на услугите отвътре в потребителския интерфейс или API слоя на приложението, има вероятност компонентите на услугите да са твърде дребнозърнести.
 - По същия начин, ако се установи, че трябва да се извършва комуникация между компонентите на услугите, за да се обработи една заявка, има вероятност компонентите на услугите да са или твърде дребнозърнести, или да не са разделени правилно от гледна точка на бизнес функционалността.
-



Съображения

- Архитектурният модел на микроуслугите решава много от общите проблеми, които се срещат както в монолитните приложения, така и в архитектурите, ориентирани към услугите.
 - Тъй като основните компоненти на приложението са разделени на по-малки, отделно внедрени единици, приложенията, изградени по архитектурния модел на микросървисите, обикновено са по-устойчиви, осигуряват по-добра мащабируемост и могат по-лесно да поддържат непрекъснатост на доставките.
-



Съображения

- Друго предимство на този модел е, че той осигурява възможност за внедрявания в реално време, като по този начин значително намалява необходимостта от традиционните месечни или уикенд “big bang” за внедрявания.
 - Тъй като промяната обикновено е изолирана до конкретни компоненти на услугата, само компонентите на услугата, които се променят, трябва да бъдат внедрени. Ако се разполага само с една инстанция на компонент на услугата, можете да се напише специализиран код в приложението за потребителски интерфейс, за да се открие активно внедряване и да се пренасочат потребителите към страница за грешка или страница за изчакване.
 - Алтернативно, можете да се разменя няколко инстанции на компонент на услугата по време на внедряване в реално време, което позволява непрекъснатата наличност по време на циклите на внедряване (нещо, което е много трудно да се направи с модела на многослойната архитектура).
-



Съображения

- Едно последно съображение, което трябва да се вземе предвид, е, че тъй като моделът за архитектура на микроуслугите е разпределена архитектура, той се сблъсква с някои от същите сложни проблеми, които се срещат в модела за архитектура, управлявана от събития, включително създаване, поддържане и управление на договори, отдалечена наличност на системата и удостоверяване и оторизиране на отдалечен достъп.
-



Архитектура „Микроядро“

Microkernel Architecture

Архитектурният модел на микроядрото (известен и като plug-in architecture) е шаблон за реализиране на приложения, базирани на продукти. Продуктово базирано приложение е такова, което е предоставено за изтегляне в различни версии. Архитектурният модел на микроядрото позволява да се добавят допълнителни функции на приложението като допълнение към основното приложение, което осигурява разширяемост, както и разделяне и изолиране на функциите.

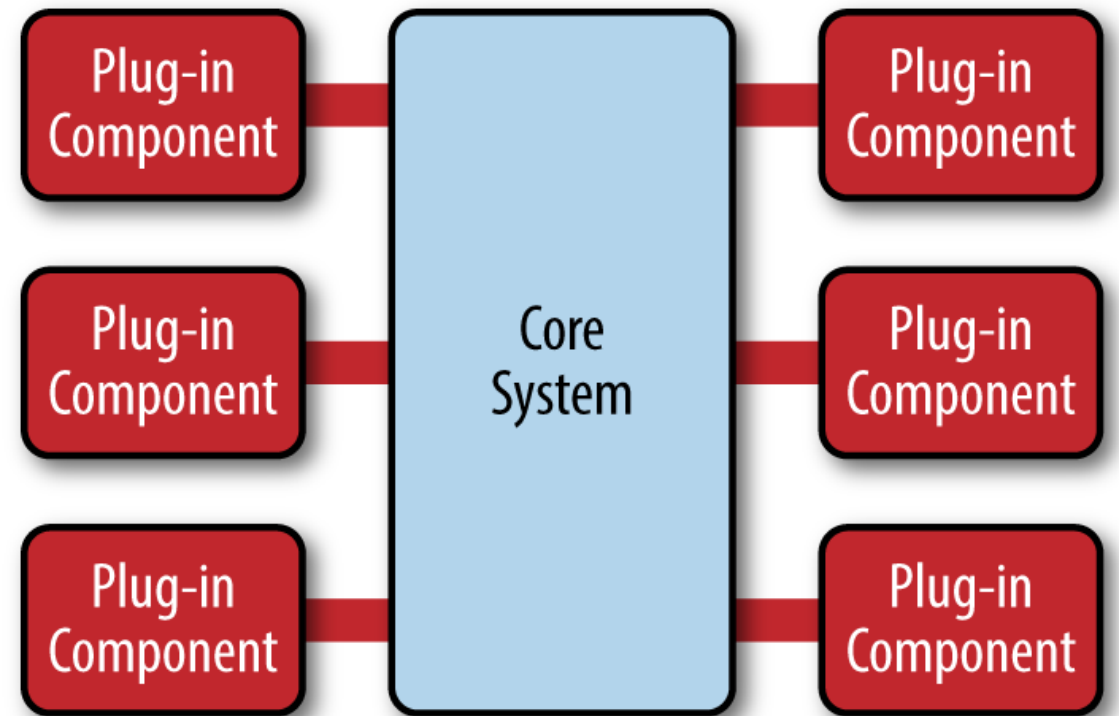




Архитектура „Микроядро“

- Архитектурният модел на микроядрото се състои от два типа архитектурни компоненти:
 - основна система (core system)
 - модули за добавяне (plug-in modules)
 - Логиката на приложението е разделена между независими plug-in модули и основното ядро на системата, като осигурява разширяемост, гъвкавост и изолация на функциите на приложението от специфичната логика.
-

Архитектура “Микроядро”





Архитектура „Микроядро“

- Основната система (core system) традиционно съдържа само минималната функционалност, необходима за функционирането на системата.
 - Много операционни системи имплементират този модел, откъдето идва и името му (micro kernel).
 - От гледна точка на бизнес приложенията основната система често е дефинирана като общата бизнес логика без специфичния код за специални случаи, специални правила или сложна условна обработка.
-



Архитектура „Микроядро“

- Плъгин модулите са самостоятелни, независими компоненти, които съдържат специализирана обработка, допълнителни функции и специфичен код, предназначени да подобрят или разширят основната система.
 - По принцип плъгин модулите трябва да бъдат независими от други плъгин модули, но може да бъдат проектирани и такива, които изискват наличието на други плъгин модули.
 - Важно е комуникацията между плъгините да бъде сведена до минимум, за да се избегнат проблеми със зависимостите.
-

Архитектура „Микроядро“

- Основната система трябва да знае кои плъгини са налични и как да стигне до тях.
- Един често срещан начин за реализиране на това е чрез някакъв вид регистър на плъгините. Този регистър съдържа информация за всеки плъгин, включително неща като име, договор за данни и подробности за протокола за отдалечен достъп (в зависимост от това как плъгинът е свързан с основната система).
- Например плъгин за данъчен софтуер, който маркира високорискови елементи за данъчни ревизии, може да има запис в регистъра, който съдържа:
 - името на услугата (AuditChecker)
 - договора за данни (входни данни и изходни данни)
 - формата на договора (XML).
- Той може да съдържа и WSDL, ако достъпът до плъгина се осъществява чрез SOAP.

Архитектура „Микроядро“

- Плъгин модулите могат да бъдат свързани към основната система по различни начини:
 - OSGi (open service gateway initiative)
 - обмен на съобщения
 - уеб услуги
 - или дори директно свързване point-to-point.
 - Видът на връзката, която се използва, зависи от:
 - вида на приложението, което се изгражда - малък продукт или голямо бизнес приложение
 - и от конкретните нужди - например единично внедряване или разпределено внедряване.
 - Самият архитектурен шаблон не уточнява нито една от тези подробности, а само това, че плъгините трябва да останат независими един от друг.
-



Архитектура „Микроядро“

- Договорите между плъгините и основната система могат да варират от стандартни до специфични договори.
 - Специфичните договори се срещат в ситуации, когато плъгините са разработени от трета страна и тогава нямате контрол върху договора.
 - В такива случаи е обичайно да се създаде адаптер между договора на плъгина и стандартния договор, използван в приложението, така че основната система да не се нуждае от специализиран код за всеки плъгин.
 - Когато се създават стандартни договори (обикновено реализирани чрез XML или Java Map), е важно да не се забравя да се създаде и стратегия за управление на версиите от самото начало.
-

Пример за архитектура „Микроядро“

- Може би най-добрият пример за архитектурата „микроядро“ е Eclipse IDE. Основната система (ядрото) е малко повече от модерен редактор. Въпреки това, след като започнат да се добавят плъгини, той се превръща изключително гъвкав и полезен продукт.
 - Интернет браузърите са друг често срещан пример за продукт, използващ архитектура “микроядро”: плъгини добавят допълнителни възможности, които иначе не се намират в основния браузър (т.е. в основната система).
-

Пример за архитектура „Микроядро“

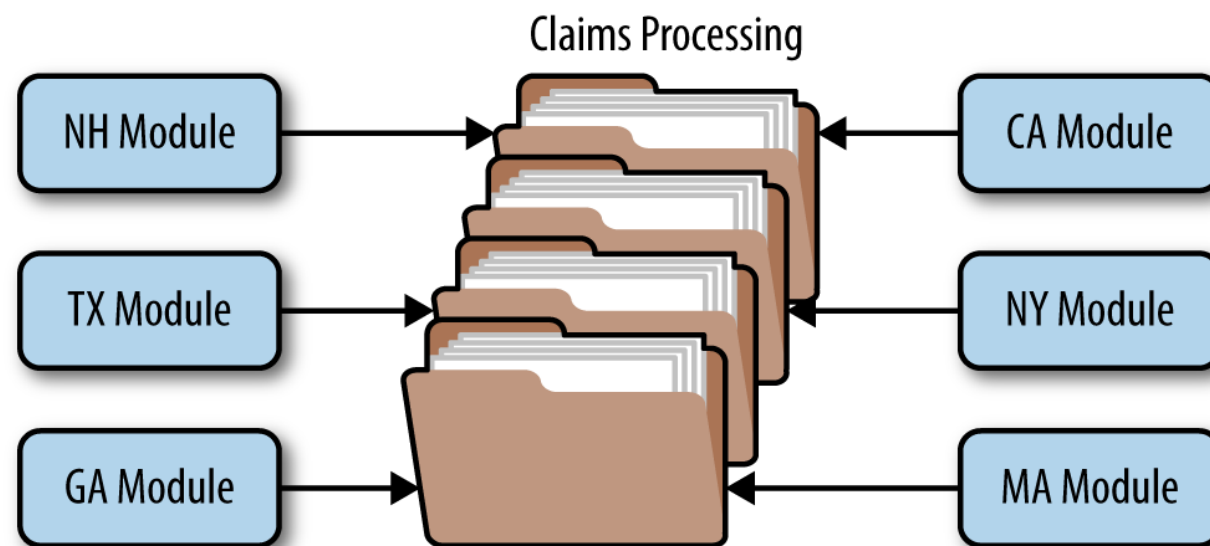
- Примерите за софтуер, базиран на продукти, са безкрайни, но какво може да се каже за големи бизнес приложения?
 - Архитектурата „микроядро“ е приложима и в тези ситуации. За да се илюстрира това, ще се използва друг пример със застрахователна компания, но този път свързан с обработката на застрахователни искове.
 - Обработката на искове е много сложен процес. Например в САЩ всеки щат има различни правила и разпоредби за това какво е и какво не е позволено в застрахователна претенция. Например, някои щати позволяват безплатна смяна на предното автомобилно стъкло, ако то е повредено от камък, докато други щати не го позволяват. Това създава почти безкраен набор от условия за стандартен процес на уреждане на претенции.
-



Пример за архитектура „Микроядро“

- Не е изненадващо, че повечето приложения за застрахователни искиове използват големи и сложни машини с правила, за да се справят с голяма част от тази сложност.
 - Тези машини с правила обаче могат да се превърнат в сложна голяма топка кал (big ball of mud), където промяната на едно правило влияе на други правила или простата промяна на правило изисква армия от анализатори, разработчици и тестери.
 - Използването на архитектурния модел на микроядрото може да реши много от тези проблеми.
-

Архитектура „Микроядро“. Пример.



Пример за архитектура „Микроядро“

- Наборът от папки представлява основната система за обработка на претенции. Тя съдържа основната бизнес логика, изисквана от застрахователната компания за обработка на претенция, само че без никаква специализирана обработка.
- Всеки плъгин модул съдържа специфичните правила за щата, в който оперира компанията.
- В този пример плъгините могат да бъдат реализирани с помощта на специализиран сорс код или отделни инстанции на машината за правила.
- Независимо от реализацията, ключовият момент е, че специфичните за щата правила и обработка са отделени от основната система за обработка на претенции и могат да бъдат добавяни, премахвани и променяни с малък или никакъв ефект върху останалата част от основната система или други плъгини.



Съображения

- Едно от предимствата на архитектурния шаблон „микроядро“ е, че той може да бъде вграден или използван като част от друг архитектурен шаблон.
 - Например, ако този шаблон решава конкретен проблем, който има с определена нестабилна област на приложението, може да се окаже, че въпреки това не може да се реализира цялата архитектура, използвайки този шаблон.
 - В този случай може да се вгради архитектурния модел „микроядро“ в друг модел, който се използва (напр. многослойна архитектура).
 - По подобен начин компонентите на процесора за събития, описани при архитектурата, управлявана от събития, могат да бъдат реализирани с помощта на архитектурния шаблон „микроядро“.
-



Съображения

- Архитектурният модел „микроядро“ осигурява добра поддръжка на еволюционно проектиране и инкрементално разработване. Първоначално може да се създаде стабилна основна система и с постепенното развитие на приложението да се добавят функции и функционалност, без да се налага да се правят значителни промени в основната система.
 - За приложения, базирани на продукти, архитектурния шаблон „микроядро“ винаги трябва да бъде първи избор като начална архитектура, особено за тези продукти, за които ще се пускат допълнителни функции с течение на времето и трябва да се контролира кои потребители какви функции получават. Ако с течение на времето се установи, че архитектурата не удовлетворява всички изисквания, винаги може да се направи рефакторинг към друг архитектурен модел, който е по-подходящ за специфичните изисквания.
-



Пространствено базирана архитектура

Space-Based Architecture



Мотивация

- Повечето уеб-базирани бизнес приложения следват един и същ общ поток на заявките:
 - заявката от браузъра се получава от уеб сървър
 - след това стига до приложния сървър
 - и накрая се обработва и от сървъра за базата данни.
- Макар че този модел работи чудесно за малък брой потребители, при увеличаване на потребителското натоварване започват да се появяват тесни места, първо на ниво уеб сървър, след това на ниво приложен сървър и накрая на ниво сървър на базата данни.
- Обичайният начин за справяне с тесните места, дължащи се на увеличаване на потребителското натоварване, е да се скалират уеб сървърите. Това е сравнително лесно и евтино и понякога работи за решаване на тези проблеми.
- В повечето случаи на високо потребителско натоварване обаче скалирането на слоя с уеб сървъри просто премества тясното място надолу към приложния сървър. Мащабирането на приложните сървъри може да бъде по-сложно и скъпо от това на уеб сървърите и обикновено просто премества тясното място надолу към сървъра за бази данни, чието мащабиране е още по-трудно и скъпо.
- Дори и да може да се мащабира базата данни, в крайна сметка се получава триъгълна топология, в която най-широката част на триъгълника са уеб сървърите (най-лесни за мащабиране), а най-малката част е базата данни (най-трудна за мащабиране).



Мотивация

- Във всяко приложение с изключително голямо натоварване от едновременни потребители базата данни обикновено е крайният ограничаващ фактор за това колко транзакции може да бъдат обработвани едновременно.
 - Въпреки че различни технологии за кеширане и продукти за мащабиране на бази данни помагат да се решат тези проблеми, остава фактът, че мащабирането на нормално приложение за екстремни натоварвания е много трудно.
 - Моделът на пространствено-базираната архитектура е специално разработен, за да адресира и реши проблемите с мащабируемостта и паралелната обработка. Той също така е полезен за приложения, които имат променливи и непредсказуеми обеми от едновременни потребители.
 - Решаването на проблема с екстремната и променлива мащабируемост по архитектурен път често е по-добър подход от опитите за увеличаване на скалирането на базата данни или вграждането на технологии за кеширане в архитектура, която не е мащабируема.
-



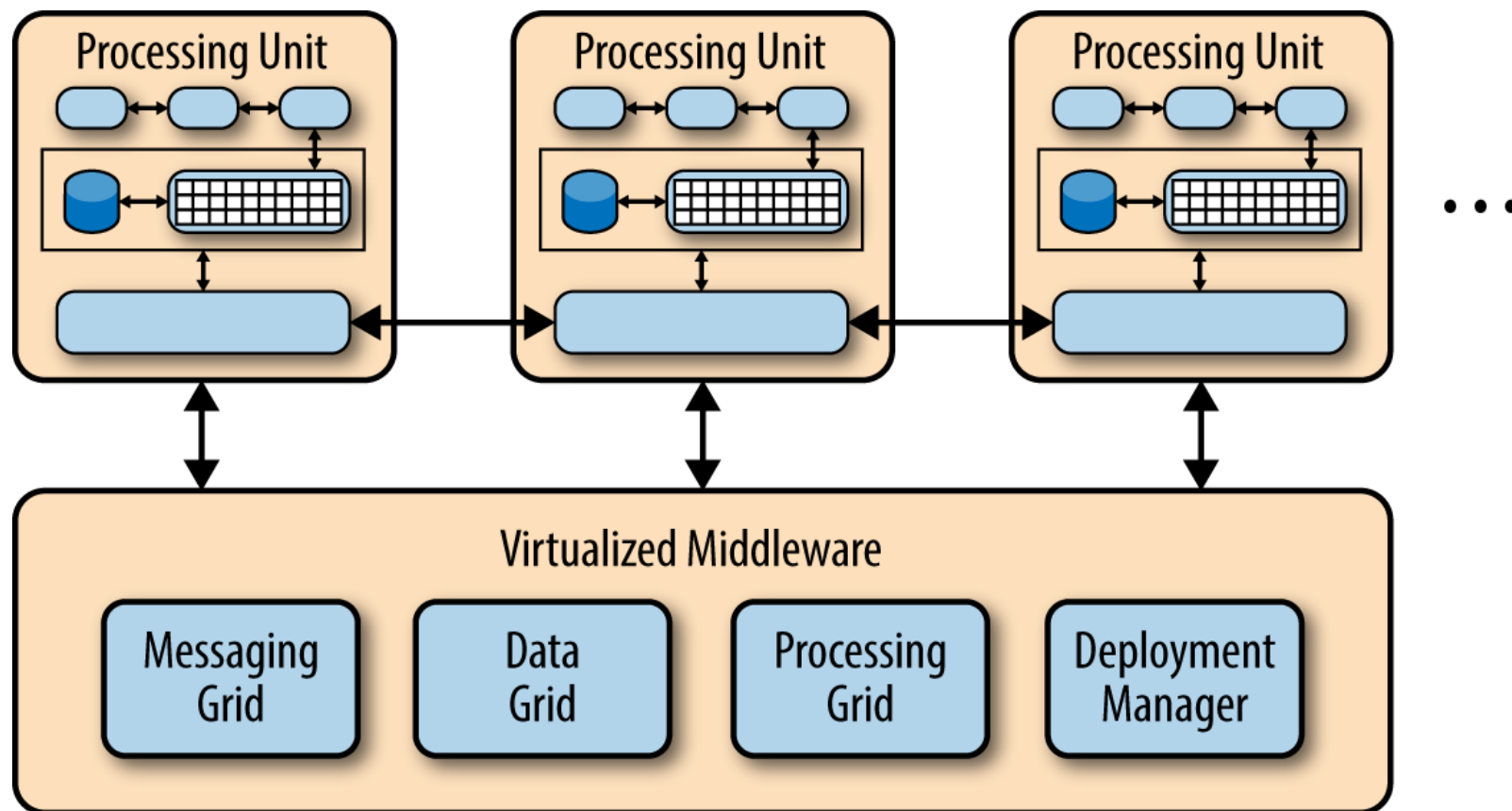
Пространствено-базирана архитектура

- Моделът на пространствено-базираната архитектура, свежда до минимум факторите, които ограничават мащабирането на приложенията.
 - Високата мащабируемост се постига чрез премахване на ограничението на централната база данни като тя се заменя с репликирани в паметта набори от данни (in-memory data grids).
 - Данните на приложението се съхраняват в паметта и се репликират между всички активни обработващи единици.
 - Обработващите единици могат да се стартират и изключват динамично, когато натоварването от потребители се увеличава и намалява, като по този начин се решава въпросът за променливата мащабируемост.
 - Тъй като няма централна база данни, тясното място на базата данни е премахнато, което осигурява почти безкрайна мащабируемост в рамките на приложението.
 - Повечето приложения, които се вписват в този модел, са стандартни уебсайтове, които получават заявка от браузъра и извършват някакво действие.
-



Пространствено-базирана архитектура

- Този архитектурен модел има два основни компонента:
 - обработващ модул (processing unit)
 - виртуализиран междинен софтуер (virtualized middleware).
 - Обработващият модул съдържа компонентите на приложението (или части от компонентите на приложението). Това включва уеб-базирани компоненти, както и бекенд бизнес логика.
 - Съдържанието на обработващия модул варира в зависимост от вида на приложението - по-малките уеб-базирани приложения вероятно ще бъдат разположени в един обработващ модул, докато по-големите приложения могат да разделят функционалността на приложението на няколко обработващ модула.
 - Обработващият модул обикновено съдържа модулите на приложението, заедно с in-memory data grid и опционално асинхронно постоянно хранилище за възстановяване при сиване на модула.
 - Също така модулът съдържа и механизъм за репликация, който се използва от виртуализирания междинен софтуер за репликация на промените в данните, направени от един обработващ модул, към други активни модули.
-



Пространственно-
базирана
архитектура



Пространствено-базирана архитектура

- Виртуализираният middleware се занимава с поддържането на реда и комуникациите. Той съдържа компоненти, които контролират различни аспекти на синхронизацията на данни и обработката на заявки.
 - Включени във виртуализирания middleware са:
 - мрежата за съобщения.
 - мрежата за данни.
 - мрежата за обработка.
 - и мениджърът за внедряване.
-



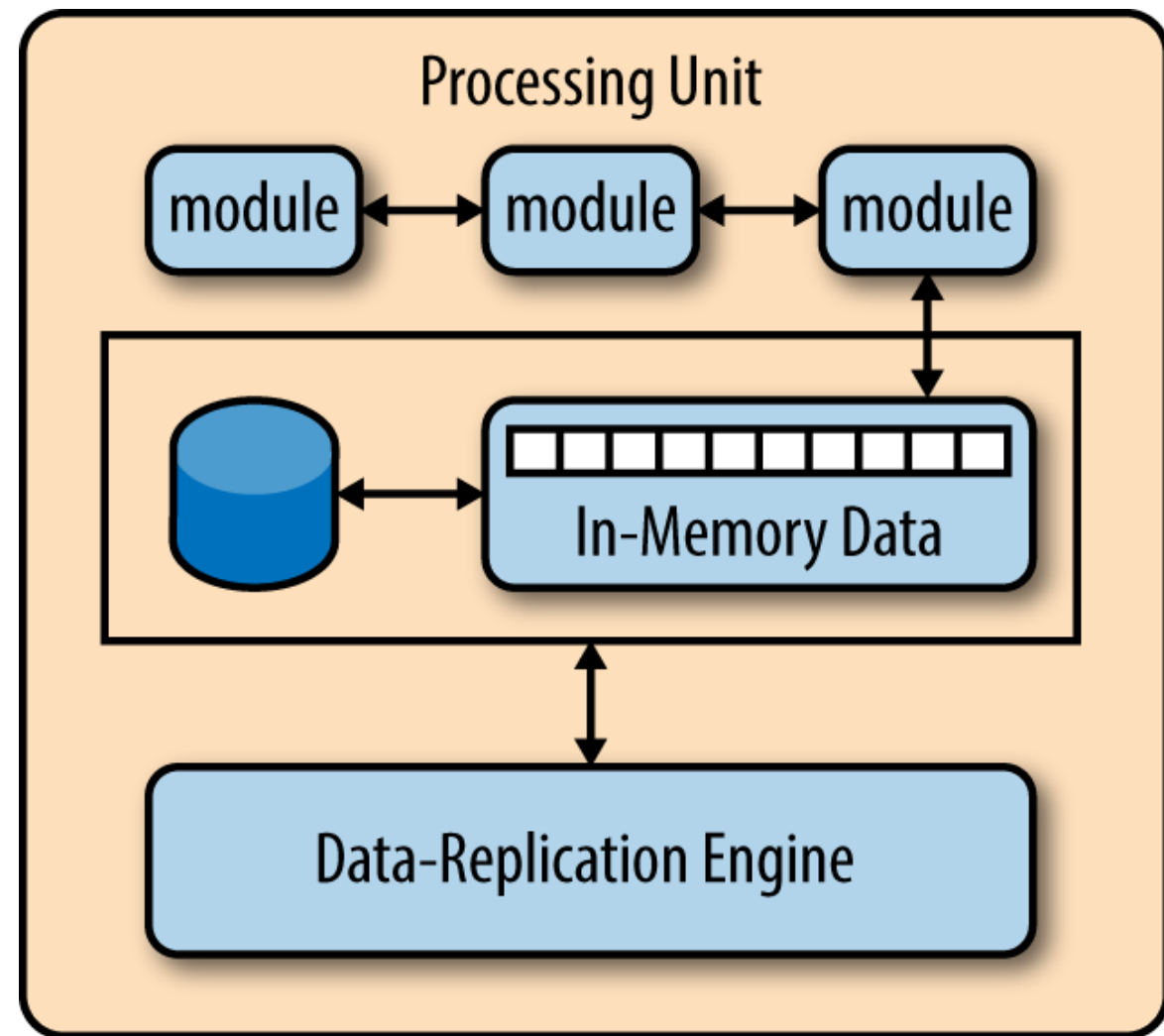
Пространствено-базирана архитектура

- Магията на пространствено-базираната архитектура се крие във виртуализирания middleware и мрежата от данни в паметта, съдържащи се във всеки обработващ модул.
 - Виртуализираният middleware е по същество контролер на архитектурата и управлява заявките, сесиите, репликирането на данни, разпределената обработка на заявките и внедряването на обработващите модули.
-

Обработващ модул

Типичната архитектура на обработващ модул съдържа:

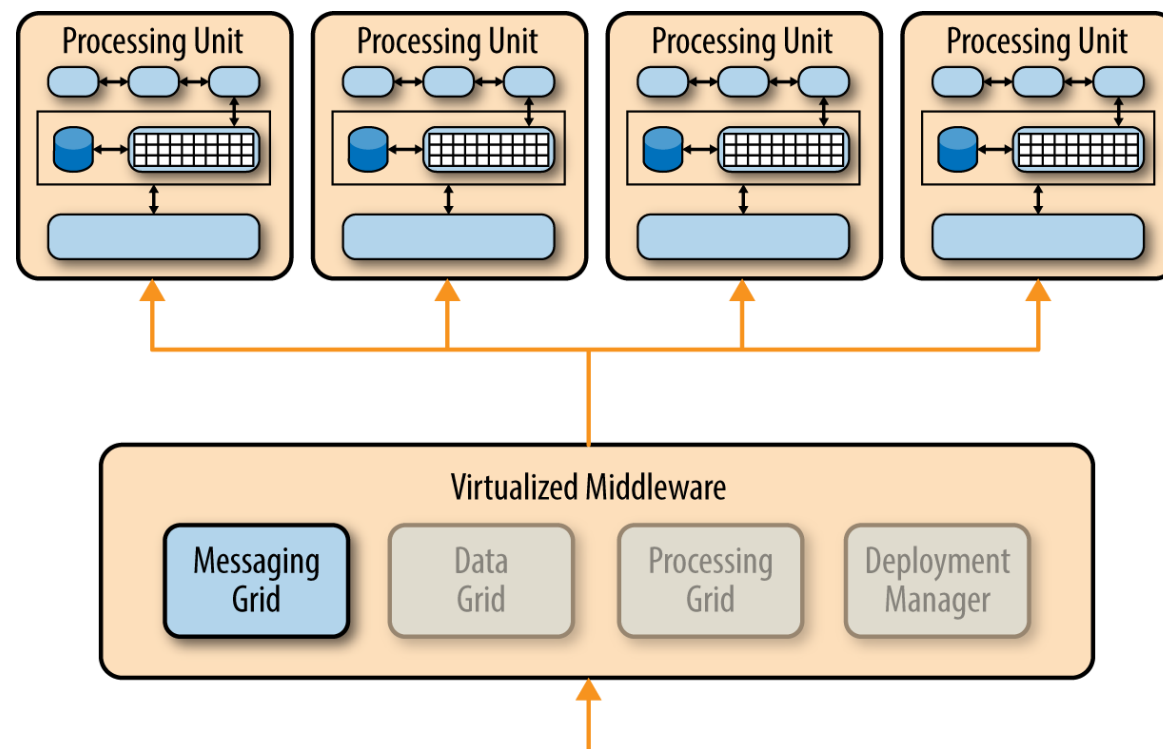
- приложните модули,
- мрежата от данни в паметта,
- незадължителното асинхронно хранилище за съхранение на данни за възстановяване при отказ
- и механизма за репликиране на данни.



Мрежа за съобщения

Messaging Grid

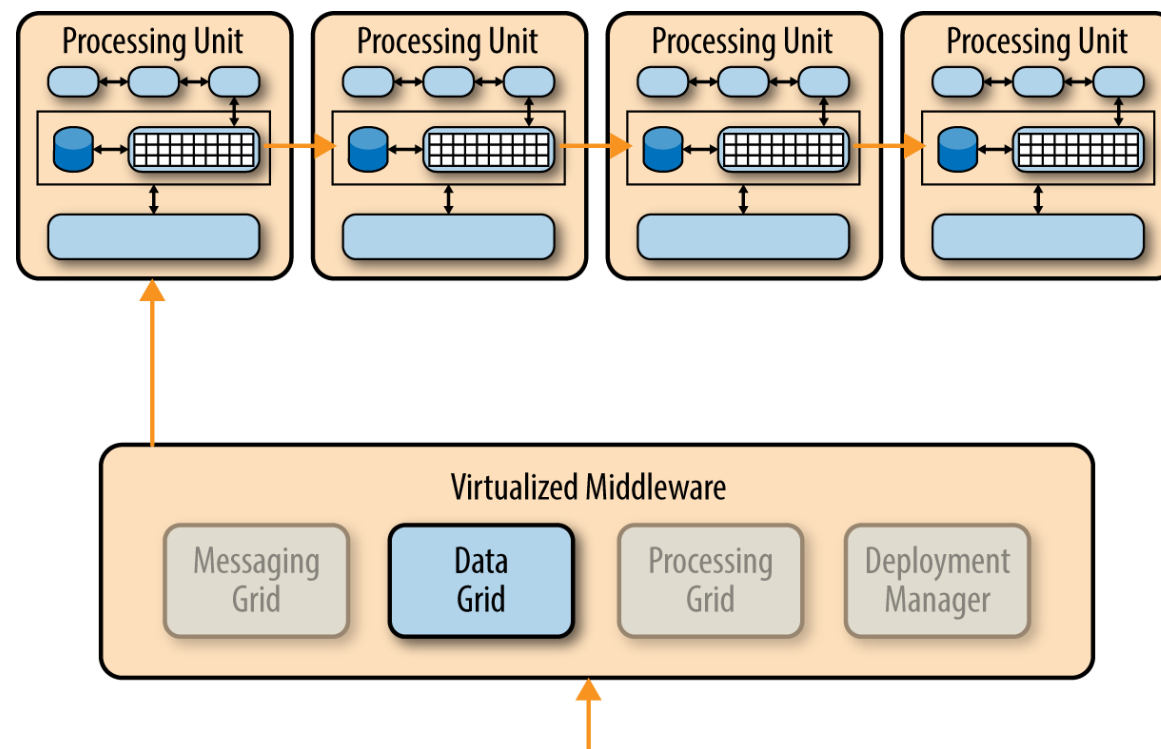
Мрежата за съобщения управлява информацията за входните заявки и сесиите. Когато във виртуализирания middleware постъпи заявка, мрежата за съобщения определя кои активни компоненти за обработка са налични, за да получат заявката, и препраща заявката към един от тях. Сложността на решетката за съобщения може да варира от по-прост round-robin алгоритъм до по-сложен алгоритъм, който следи коя заявка от кой модул се обработва.



Мрежа за данни

Data Grid

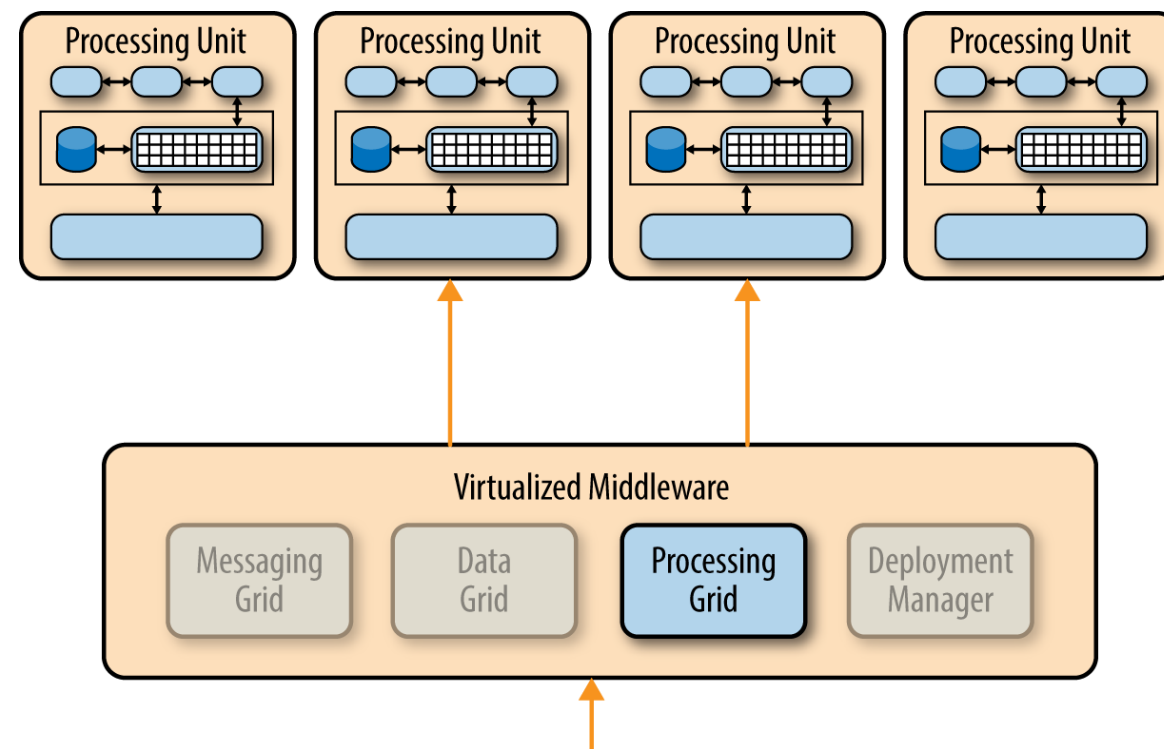
Компонентът на мрежата за данни е може би най-важният и решаващ компонент в този модел. Той взаимодейства с машината за репликиране на данни във всеки обработващ модул, за да управлява репликирането на данни между тях при актуализиране на данните. Тъй като мрежата за съобщения може да препрати заявка към всеки от наличните обработващи модули, от съществено значение е всеки модул да съдържа точно същите данни в своята мрежа за данни в паметта. Въпреки че на фигурата е показано синхронно репликиране на данни между обработващите модули, в действителност това се извършва паралелно, асинхронно и много бързо, като понякога синхронизирането на данните се извършва за няколко микросекунди (една милионна част от секундата).



Мрежа за обработка

Processing Grid

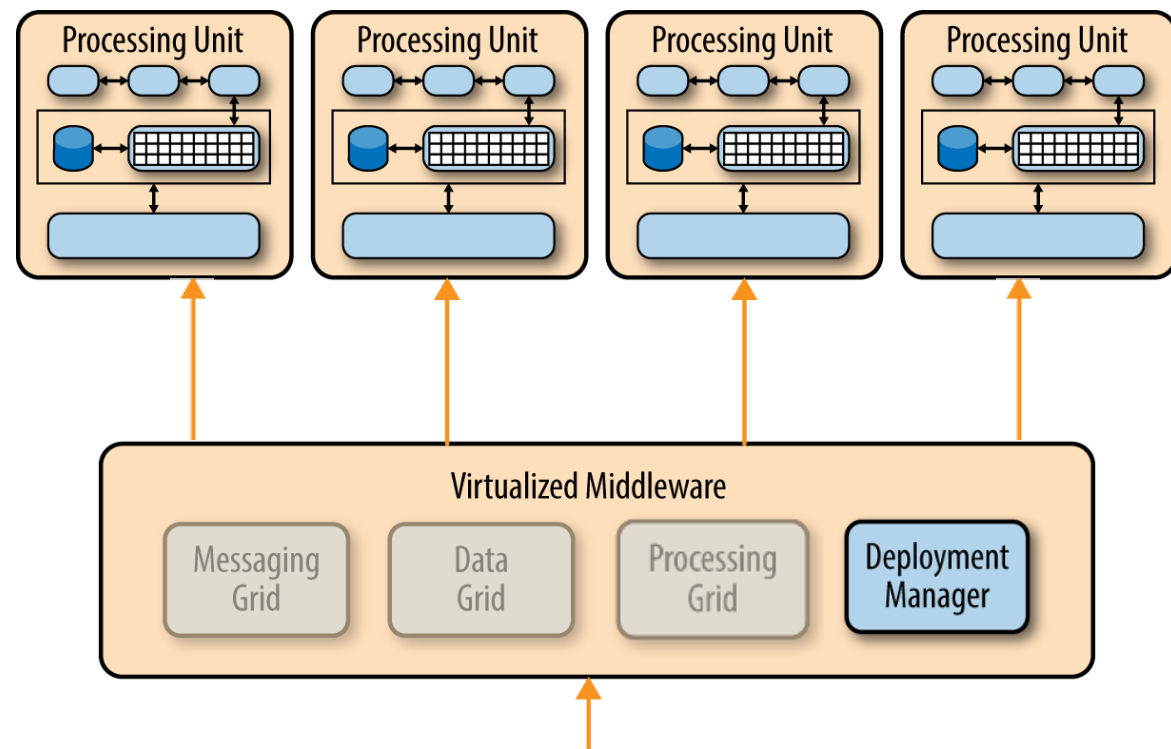
Мрежата за обработка е незадължителен компонент във виртуализирания middleware, който управлява разпределената обработка на заявките, когато има множество обработващи модули, всеки от които съдържа част от имплементацията на логиката на приложението. Ако постъпи заявка, която изисква координация между различни видове обработващи модули (например, модул за обработка на поръчки и модул за обработка на клиенти), именно мрежата за обработка е тази, която посредничи и оркестрира заявката между тези два обработващи модула.



Мениджър за внедряване

Deployment Manager

Компонентът за управление на внедряването управлява динамичното стартиране и изключване на обработващи модули в зависимост от натоварване. Този компонент непрекъснато следи времето за отговор и потребителското натоварване и стартира нови обработващи модули, когато натоварването се увеличи, и ги изключва когато натоварването намалее. Това е критичен компонент за постигане на променливи нужди от мащабируемост в рамките на приложението.





Съображения

- Пространствено-базираният архитектурен модел е сложен и скъп за изпълнение. Той е добър архитектурен избор за малки уеб базирани приложения с променливо натоварване (например сайтове за социални медии, сайтове за търгове и аукциони).
 - Той обаче не е подходящ за традиционно големи приложения с релационни бази данни с големи количества оперативни данни.
-



Съображения

- Много от архитектурните компоненти в този модел може да се реализират чрез продукти на трети страни, като GemFire, JavaSpaces, GigaSpaces, IBM Object Grid, nCache и Oracle Coherence.
 - Тъй като реализацията на този модел варира значително по отношение на разходите и възможностите (особено времето за репликация на данни), архитектът трябва първо да установи какви са специфични цели и нужди, преди да направи избор на продукт.
-